程序设计基础及语言 —— C++ 大学教程

第 6 章　函数和递归入门

邱 远

东南大学 网络空间安全学院
yuanqiu@seu.edu.cn

2026 年 11 月

目录

函数与类型转换

随机

说明符 (static, extern, inline)、存储类别、函数调用栈

拷贝与引用

函数重载、默认参数、函数模板

递归与迭代

回顾：成员函数

https://en.cppreference.com/cpp/language/member_functions

```
6   /** @class GradeBook
7    ** @brief First definition of class */
8   class GradeBook {
9   public:
10      /** @brief Display Welcome message */
11      void displayMessage() const {
12          std::cout << "Welcome to the Grade Book!" << std::endl;
13      } // end function
14  }; // end class
```

回顾：C++ 函数

返回类型 函数名(输入参数) { 函数体; }

C++ 模块化编程：类、函数、命名空间、……

模块化编程

模块化编程 (modular programming)

将大程序拆分为负责不同功能的模块，分而治之

- 采用分治法，使程序由小到大构建，易于管理

```
fridge.open();
put(elephant, fridge);
fridge.close();
```

- 代码复用
- 避免重复代码

回顾

- 结构化编程：避免`goto`，使用控制结构
- 面向对象的编程：将数据与对数据的操作封装在同一个类中

面向对象/过程的编程都是模块化编程！

已出现过的函数

- 第 1 章 (本章中介绍)
  - `time_t time(time_t*);` // <ctime> 返回当前时间
  - `void srand(int);` // <cstdlib> 设置随机种子
  - `int rand();` // <cstdlib> 生成随机整数
- 第 2 章
  - `int main();`
- 第 3 章
  - `void GradeBook::displayMessage() const;`
  - `istream& getline(istream&,string);` // <string>读取一行
  - `size_t string::size() const;` // <string>
  - `size_t string::length() const;` // <string>
  - `string string::substr(size_t, size_t) const;` // <string>
- 第 5 章
  - `double pow(double, double);` // <cmath>
  - `int istream::get();` // <iostream>读取一个字符
- 此外，所有学过的运算符都可以理解为函数
  - `bool operator&&(bool, bool);`

<cmath> 库中的其他函数

https://cplusplus.com/reference/cmath/

- 取整 (注意取整函数返回类型为double)
  - `double ceil(double x);` 返回 $\geq x$ 的最小整数
  - `double floor(double x);` 返回 $\leq x$ 的最大整数
  - `double round(double x);` 四舍五入，± 0.5 返回离 0 远的
- 指数对数
  - `double exp(double x);` 返回 e^x
  - `double pow(double base, double exponent);` 返回 $base^{exponent}$
  - `double sqrt(double x);` 返回 $\sqrt{x}$
  - `double log(double x);` 返回 $\ln x$
  - `double log10(double x);` 返回 $\log_{10} x$
- 三角函数 (使用弧度值)
  - `double sin(double x);` 返回 $\sin(x)$
  - `double cos(double x);` 返回 $\cos(x)$
  - `double tan(double x);` 返回 $\tan(x)$
- 浮点数版函数
  - `int abs(int); // <cstdlib>` 返回 $|x|$
  - `double fabs(double); // <cmath>` 返回 $|x|$
  - `double fmod(double, double); // <cmath>` 求模

程序一：具有多个形参的函数

```
1  #include <iostream>
2  int maximum(int x, int y, int z) {
3      int maxValue = x;
4      if (y > maxValue)
5          maxValue = y;
6      if (z > maxValue)
7          maxValue = z;
8      return maxValue;
9  }
10 int main() {
11     int grade1, grade2, grade3;
12     std::cin >> grade1 >> grade2 >> grade3;
13     std::cout << "Max grade: " << maximum(grade1, grade2, grade3) << std::endl;
14 }
```

- 函数定义 (Line 2) 包括返回类型、函数名、形参列表、函数体
- 函数调用 (Line 13) 使用函数名和实参列表
- 实参列表不是逗号运算符，计算顺序不固定，代码不应依赖其顺序

```
maximum(x++, x+y, y); // do not rely on argument order
```

Line 8：返回语句

```
2  int maximum(int x, int y, int z) {
3      int maxValue = x;
4      if (y > maxValue)
5          maxValue = y;
6      if (z > maxValue)
7          maxValue = z;
8      return maxValue;
9  }
```

- 函数执行至return语句时，会直接返回函数调用处
  - 任何后续语句、未结束的循环等都会被忽略，从调用处继续执行
- 若函数返回类型为void，则没有返回值，可以通过return;返回，或在执行至函数体右大括号时自动返回
- 否则，必须通过return 表达式;返回，并且表达式需要能转换为返回类型

函数原型与函数签名

- 和变量一样，函数也必须先声明后使用
- 我们在第 3 章介绍过，函数声明 (也称函数原型, function prototype) 包含如下信息
  - 返回类型
  - 函数名
  - 形参类型列表
  - 分号[1]
- C++ 允许同名函数，因此无法通过函数名唯一确定所调用的函数

函数签名 (function signature)

函数原型中的函数名 + 形参类型列表构成函数签名

- 确定函数后，返回类型也即确定，因此返回类型不构成签名
- 同一个作用域内，不允许有两个签名一致的函数

```
int findMax(int x, int y);
void findMax(int x, int y); // error: cannot be overloaded
```

[1] 若为大括号，则这是一个函数定义

程序二：实参类型隐式转换

```
1  #include <iostream>
2
3  double power(double, unsigned int);
4
5  int main() {
6      std::cout << power(5, 2) << std::endl;
7  }
8
9  double power(double x, unsigned int n) {
10     double res{1.};
11     for (unsigned int i = 0; i < n; ++i) {
12         res *= x;
13     }
14     return res;
15 }
```

- 虽然power函数声明需要类型(double, unsigned int)，我们调用时使用两个整数也能够计算
- 这是因为发生了实参隐式转换 (implicit conversion)

回顾：显式类型转换

https://en.cppreference.com/cpp/language/static_cast

```
25        double average = static_cast<double>(total) / counter;
```

显式类型转换

`static_cast<目标类型>(变量名);`

- 整数之间的除法是整除，只有一方为浮点数时才能调用小数除法
- 有时类型转换是隐式的，例如
  - `double average = total; average = average / counter;`
  - `double average = total * 1. / counter;`
  - `if (counter)`
  - 显式转换类型是一个好习惯
- 也可以通过转换类型来避免溢出
  - `long sum = static_cast<long>(num1) + num2;`
- 二进制下某些小数是不精确的（如 $\frac{1}{3}$），因此通常不要做 double 之间是否相等的判断
  - `2. / 3 == 1 - 1. / 3; // false`

隐式类型转换

https://en.cppreference.com/cpp/language/implicit_conversion

隐式类型转换发生在以下场景中

- 初始化 `int n = 1L;`
- 赋值 `n = 2.1;`
- 调用函数 `power(5,2);`
- 返回语句 `return 1.0;`
- 算术、比较等运算 `avg = total * 1. / count;`

升级 (Promotion)

在不丢失数据的前提下，将一种类型转换为另一种类型称为**升级**

窄式转换 (Narrowing conversion)

产生数据丢失，或超出目标类型范围的类型转换称为**窄式转换**

⚠ Clang-Tidy: Narrowing conversion from 'unsigned int' to signed type 'int' is implementation-defined :5

图 1: 窄式转换应使用显式转换并确保逻辑正确，编译器会通过 Warning 提醒

整数转换规则

https://en.cppreference.com/cpp/language/implicit_conversion

- 整数升级 (Integer promotion)
  - 级别依次为`bool` < `char` < `short` < `int` < `long` < `long long`
  - 无符号`unsigned`与对应有符号级别相同
  - 任何级别在`int`及以下的类型，计算时都会转为`int`或`unsigned int`
- 布尔转换 (Boolean conversion)
  - 结果为 0 的对应`false`，否则为`true`
- 整数转换 (Integer conversion)
  - 任意两个整数类型可以互转，规则如下 (除整数升级与布尔转换)
  - 如果值在目标类型范围内，则不会改变
  - 否则，转为无符号类型时，不断加减其范围直至在范围内（取模）

```cpp
#include <iostream>
int main() {
    unsigned char n = -123456;
    std::cout << (static_cast<int>(n) == -123456 % 256 + 256) << std::endl;
}
```

  - 否则，转为有符号类型取决于实现（不建议使用）

浮点数转换规则

https://en.cppreference.com/cpp/language/implicit_conversion

- 浮点转整数 (Floating-integer conversion)
  - 任意浮点类型可转任意整数类型，规则如下 (除布尔转换)
  - 舍弃小数部分[2]
  - 若结果在目标类型范围内，则使用该值，否则取决于实现
- 整数/浮点转浮点 (Integer/floating-floating conversion)
  - 任意整数/浮点类型可转任意浮点类型
  - 如果能用目标类型精确表示，则使用该值
  - 如果在范围内但不能精确表示，则使用近似值，具体值取决于实现
  - 如果不在范围内，是未定义行为

二进制下，`float`为小数点后 23 位，`double`为小数点后 52 位，32 位的`int`介于二者之间，因此可以升级为`double`

- 混合表达式中按计算顺序依次升级，不会影响原值

```
int x{2}, y{3};
std::cout << (x / 1. + y / 2) / 2 << std::endl; // output 1.5
```

[2]即向 0 靠近，注意和`floor`函数的区别

目录

函数与类型转换

随机

说明符 (static, extern, inline)、存储类别、函数调用栈

拷贝与引用

函数重载、默认参数、函数模板

递归与迭代

标准库

https://cppreference.com/cpp/standard_library

C++ 标准库提供常用的功能，我们已经学习了其中一些

- <iostream> 标准流输入输出，如cin，cout
- <iomanip> 有参数的流操作算子
- <string> 字符串
- <cmath> 数学函数
- <limits> 类型范围

我们将在余下的课程中接触一些其他的头文件，本章新增

- <cstdlib> C 语言标准库
- <ctime> C 语言日期时间
- <random> 随机 (C++11 起)

程序三：生成随机数

```
1  #include <cstdlib>
2  #include <iomanip>
3  #include <iostream>
4
5  int main() {
6      for (unsigned int counter = 0; counter < 20; ++counter) {
7          std::cout << std::setw(10) << (1 + std::rand() % 6);
8          if (counter % 5 == 4)
9              std::cout << std::endl;
10     }
11 }
```

程序输出

```
         6         6         5         5         6
         5         1         1         5         3
         6         6         2         4         2
         6         2         3         4         1
```

Line 7：rand 函数

https://en.cppreference.com/cpp/numeric/random/rand

```
7         std::cout << std::setw(10) << (1 + std::rand() % 6);
```

- C 标准库<stdlib.h>在 C++ 中通过#include <cstdlib>使用
  - C 语言的头文件比较特殊，允许进入全局命名空间[3]
  - #include <stdlib.h>保证提供::rand()，可能提供std::rand()
  - #include <cstdlib>保证提供std::rand()，可能提供::rand()
  - 因此rand不带std::也许可以编译，但不建议使用
- int rand();返回 $[0, \text{RAND_MAX}]$ 间的随机整数
  - RAND_MAX取决于实现，如INT_MAX或0x7fff
  - rand() % x 近似于 $[0, x-1]$ 上的随机整数，因此 a + rand() % (b - a + 1)常用于生成 $[a, b]$ 的随机数
  - rand() * 1. / RAND_MAX可得到 $[0, 1]$ 的随机小数[4]

程序三每次执行结果都相同，并未达到真正的随机

[3] https://en.cppreference.com/cpp/header

[4] 根据逆变换采样原理 (inverse transform sampling)，可用于对任何概率分布采样

真随机与伪随机

真随机数 (true random numbers)

通过不可预测的物理过程（如热噪声、光电效应、量子现象）产生的随机数，完全无法预测

- `https://www.random.org/`通过大气噪声生成真随机数
- 计算机程序中，真随机数
  - 难以获得，需要特殊硬件支持
  - 并非必须，许多场景只要近似随机即可
  - 无法复现，难以调试和排查问题
- 计算机程序采用伪随机 (pseudorandom)[5]，由种子 (seed) 控制
  - 同样的种子会产生一致的随机数序列
  - 不同的种子产生的序列不同
  - 无法根据已产生的随机数推断种子或下个数
  - 将种子设置为当前时间等不能控制的值，达到近似真随机的效果

[5]我们此处讨论的真/伪随机是指随机数生成方式，目标依然是生成独立且均匀分布的数字。有时人们也把特定的数据分布称为“伪随机”，如“每抽一次概率变大”，或“歌手均匀分布的歌单”，注意不要混淆。

程序四：通过 srand 设置种子

https://en.cppreference.com/cpp/numeric/random/srand

```cpp
#include <cstdlib>
#include <ctime>
#include <iomanip>
#include <iostream>

int main() {
    std::srand(std::time(0));
    for (unsigned int counter = 0; counter < 20; ++counter) {
        std::cout << std::setw(10) << (1 + std::rand() % 6);
        if (counter % 5 == 4)
            std::cout << std::endl;
    }
}
```

- <cstdlib>中的void srand(unsigned);设置rand()使用的种子
  - 只需要在使用rand()前设置一次，整个随机数序列都会变化
  - 调试时可使用同样的种子来重复结果
- <ctime>中的time_t time(time_t*);返回当前 UNIX 时间
  - UNIX 时间是从 UTC 1970 年 1 月 1 日 00:00:00 至今的秒数
  - time_t没有严格规定，但通常为long，保险起见可以显式转换
    std::srand(static_cast<unsigned>(std::time(0)));
  - 学习指针前，使用std::time(0)即可

random 头文件

https://en.cppreference.com/cpp/header/random

C++11 引入了<random>头文件，更推荐通过它生成随机数

```
1  #include <iomanip>
2  #include <iostream>
3  #include <random>
4
5  int main() {
6      std::random_device rd; // object rd generates seeds
7      std::default_random_engine engine{rd()}; // object engine initialized with a seed
8      std::uniform_int_distribution<unsigned> randInt{1, 6}; // object randInt generates random ints in [1,6]
9      for (unsigned int counter = 0; counter < 20; ++counter) {
10         std::cout << std::setw(10) << randInt(engine);
11         if (counter % 5 == 4)
12             std::cout << std::endl;
13     }
14 }
```

练习：C++ 实现 21 点

小练习

用 C++ 实现简化的 21 点游戏，规则如下

- 每张牌点数为 {A,2-10,J,Q,K} 之一
- 每次发牌任意点数概率均为 1/13
- A 可视为 1 或 11，{J,Q,K} 视为 10
- 先向玩家发两张牌 (输出)，并询问玩家是否要牌，发牌至玩家停牌或爆牌
- 若点数超过 21 点，则玩家爆牌 (bust)，输出玩家失败
- 若点数为 21 点 (blackjack)，输出玩家胜利
- 若玩家未达 21 点即停牌，为电脑发牌直到超过玩家点数
- 若电脑爆牌则玩家胜利，否则玩家失败

练习：C++ 实现 21 点

```cpp
#include <iostream>
#include <string>
#include <random>

class Card {
    int id;
public:
    explicit Card(int id_): id(id_) {}
    std::string to_string() const {
        switch (id) {
            case 1: return "A";
            case 10: return "10";
            case 11: return "J";
            case 12: return "Q";
            case 13: return "K";
            default: return {static_cast<char>(id - 1 + '1')}; // std::string constructor
        }
    }
    int value() const {
        switch (id) {
            case 1: return 11;
            case 11: case 12: case 13: return 10;
            default: return id;
        }
    }
    bool is_A() const { return id == 1; }
};
// continued on next page
```

练习：C++ 实现 21 点

```cpp
int main() {
    // 1. deal player card
    std::random_device rd;
    std::default_random_engine engine(rd());
    std::uniform_int_distribution<int> randInt{1, 13};
    Card card1{randInt(engine)}, card2{randInt(engine)};
    int player_value{card1.value() + card2.value()}; // count A as 11 first, recount if bust
    int player_a_count{card1.is_A() + card2.is_A()};
    std::cout << "Your Cards: " << card1.to_string() << ' ' << card2.to_string() << std::endl;

    // 2. player next card
    char input{};
    while (player_value < 21 && input != 'n' && input != 'N') {
        std::cout << "Current value is " << player_value << ", next card (Y/N)?";
        std::cin >> input;
        if (input == 'y' || input == 'Y') {
            Card card{randInt(engine)};
            std::cout << "You got a new card: " << card.to_string() << '\n';
            player_value += card.value();
            player_a_count += card.is_A();
            // recount A for 1 if bust
            while (player_value > 21 && player_a_count > 0) {
                --player_a_count;
                player_value -= 10;
            }
        }
    }
    // continued on next page
```

练习：C++ 实现 21 点

```
57      if (player_value == 21) {
58          std::cout << "Blackjack! You win!\n";
59      } else if (player_value > 21) {
60          std::cout << "Bust! You lose!\n";
61      } else {
62          int computer_value{}, computer_a_count{};
63          while (computer_value <= player_value) {
64              Card card{randInt(engine)};
65              computer_value += card.value();
66              computer_a_count += card.is_A();
67              while (computer_value > 21 && computer_a_count > 0) { // duplicated code
68                  --computer_a_count;
69                  computer_value -= 10;
70              }
71              std::cout << "Computer got a new card: " << card.to_string() << ", total = " << computer_value <<'\n';
72          }
73          if (computer_value > 21) {
74              std::cout << "Computer bust! You win!\n";
75          } else {
76              std::cout << "Computer win! You lose!\n";
77          }
78      }
79  }
```

目录

函数与类型转换

随机

说明符 (static, extern, inline)、存储类别、函数调用栈

拷贝与引用

函数重载、默认参数、函数模板

递归与迭代

说明符

- 说明符 (Specifier) 用于修饰变量、函数、类等，用来改变其性质
- 有时也称修饰符 (Modifier)
- 我们已经学过部分说明符
  - `unsigned`, `signed`用于说明整型是否有符号
  - `const`, `constexpr`用于声明常量
    - `const`还可用于成员函数，说明不会修改数据成员
  - `public: private:`用于控制类成员的访问
  - `explicit`阻止隐式转换类型
- 我们在本章学习存储类别说明符`static`，`extern`以及函数说明符`inline`

存储期、作用域、链接

- 我们在第 3 章对比过全局变量、局部变量、数据成员的作用域
- 本章中，我们对**存储期**、**作用域**、**链接**做详细介绍
- 我们主要讨论局部变量、全局变量、全局函数，后续学到的命名空间变量与函数和全局变量与函数几乎一致

存储期 (storage duration)

决定变量在内存中存在的时间

作用域 (scope) 与链接 (linkage)

共同决定变量在哪些地方可见

- 作用域：在代码的哪些区域可见
- 链接：在其他文件中是否可见

存储期

https://en.cppreference.com/cpp/language/storage_duration

C++ 中有 4 种存储期类型

- 自动存储期 (automatic storage duration)
  - 未加任何说明[6]的**局部变量与函数形参**为自动存储期
  - 它们会在声明时创建，在作用域结束时 (对应的 }) 自动销毁

```
int inc(int x) { // automatic variable x
    return x + 1;
} // x is destroyed
```

- 静态存储期 (static storage duration)
  - **全局变量与使用`static`局部变量**为静态存储期
  - 它们会在程序开始时创建，在程序结束时销毁
- 动态存储期 (dynamic storage duration)
  - (后续章节) `new`时创建，`delete`时销毁
- 线程存储期 (thread storage duration)
  - (后续章节) 线程结束时销毁

[6] C++ 曾经用于存储期的关键词`auto`与`register`均已弃用

static 说明符（局部变量）

https://en.cppreference.com/cpp/keyword/static

static说明符有许多用处，建议统一理解为“只有一个”

- 用于**局部变量**时，说明该变量为**静态存储**
- 表现类似于全局变量，只是被放在了函数中
- 在程序开始时创建，直到结束才会销毁
- 全局变量和静态变量不初始化也会为 0，但依然应该初始化

```cpp
#include <iostream>
int global_var{1}; // only initialize once
void increment() {
    int auto_var{}; // initialize every function call
    static int static_var; // only initialize once
    ++auto_var; ++static_var; ++global_var;
    std::cout << auto_var << ' '
              << static_var << ' '
              << global_var << std::endl;
} // only auto_var is destroyed
int main() {
    increment(); // 1 1 2
    increment(); // 1 2 3
    increment(); // 1 3 4
}
```

static 说明符（数据成员/拓展）

https://en.cppreference.com/cpp/keyword/static

static说明符有许多用处，建议统一理解为“只有一个”

- 用于类的**数据成员**时，该类的所有对象“共用一个”成员
- 表现类似于全局变量，只是被放在了类中

```
#include <iostream>
class MyClass {
    static int static_member; // declaration in .h file (pre C++17)
    const static int const_static_member{1};
public:
    void increment() {
        std::cout << ++static_member << ' '
                  << const_static_member << '\n';
    }
};
int MyClass::static_member = 0; // definition in .cpp file
int main() {
    MyClass obj1;
    obj1.increment(); // 1 1
    MyClass obj2;
    obj2.increment(); // 2 1
}
```

static 说明符（成员函数/拓展）

https://en.cppreference.com/cpp/keyword/static

static说明符有许多用处，建议统一理解为“只有一个”

- 用于类的**成员函数**时，该函数可以直接使用而无需构建对象，但不能访问非static的类成员 (没有对象)
- 表现类似全局函数，只是被放在了类中

```
#include <iostream>
class MyClass {
    static int static_member; // declaration in .h file (pre C++17)
    const static int const_static_member{1};
public:
    static void increment() {
        std::cout << ++static_member << ' '
                  << const_static_member << '\n';
    }
};
int MyClass::static_member = 0; // definition in .cpp file
int main() {
    MyClass::increment(); // 1 1
    MyClass obj;
    obj.increment(); // 2 1
}
```

作用域

https://en.cppreference.com/cpp/language/scope

C++ 中有多种作用域类型

- 局部作用域 (local scope)
  - 也称块作用域 (block scope)，变量只能在定义它的大括号内被访问
  - **局部变量**、**函数形参**、`for`的初始化等都属于块作用域
- 全局作用域 (global scope)
  - 也称文件作用域 (file scope)，变量定义后就能在整个文件中访问
  - **全局变量**、**全局函数**为全局作用域，`static`局部变量为局部作用域
- 函数原型作用域 (function prototype scope)
  - 函数原型不需要写形参名，因此其中形参名只作用于括号内
- 类作用域 (class scope)
  - 类内的数据成员具有类作用域，在类内无需解析就可调用，类外需要解析，如`MyClass::member`或`obj.member`;
- 命名空间作用域 (namespace scope)
  - 类似于类作用域，在命名空间外需要解析，如`std::cout`

函数可以是全局、类、命名空间作用域，但没有局部作用域的函数（不能在函数中定义函数，因为函数所在的代码块是它自己）

作用域解析运算符

- ::运算符用于指定全局、命名空间、类中的名称
  - 优先级最高，前后不加空格
- 若前面为空，::x只会在全局作用域找
- 若前面为命名空间或类，A::x只会在命名空间/类作用域找
- 若不加::，则寻找顺序为从当前作用域开始逐层向外找直到全局作用域，发现任何匹配的名称即停止
  - using namespace的命名空间与全局作用域在同一层（重名会歧义）

```
#include <iostream>
int x{1}; // global scope
namespace A {
    int x{2}, y{2};
    class C {
    public:
        static const int x{3};
        int y{3};
    };
}
int main() {
    {   int x{4}; // local scope >> global scope
        std::cout << x << ::x << A::x << A::C::x << std::endl; // 4123
    }{  using namespace A;
        C obj;
        std::cout << y << ::A::y << obj.y  << std::endl; // 223
    }
}
```

链接

https://en.cppreference.com/cpp/language/storage_duration

C++ 中有 3+ 种链接类型

- 无链接 (no linkage)
  - 变量只能在其作用域内访问，作用域外可重新定义同名变量
  - 局部作用域的变量都是无链接 (包括static)

```
for (int x = 1; x < 10; ++x) { ... }
int x = 2;
```

- 内部链接 (internal linkage)
  - 变量或函数能在文件内访问，其他文件可重新定义同名变量函数
  - 使用static说明[7]的**全局变量与全局函数**为内部链接
- 外部链接 (external linkage)
  - 变量或函数能在任何文件中访问，只能定义一次，其他文件中声明
  - 未用static**全局变量与全局函数**为外部链接，类的**成员函数定义**、**静态数据成员定义**为外部链接
  - 外部链接可理解为“出口”，即别人能从这个.cpp里用什么
- 模块链接 (module linkage)：C++20 引入

[7]注意和static之前的作用区分：由于全局变量/函数已经是静态的了，不需要由static说明其“只有一个”，因此含义变成“只在此文件”

static 与 extern 说明符（全局变量）

https://en.cppreference.com/cpp/keyword/extern

- 由于全局变量已经有明确的存储期（静态）与作用域（全局），只需要对其链接做出说明
- 全局变量与全局函数默认为外部链接，通过static说明内部链接
- 全局常量默认为内部链接，通过extern说明外部链接
- 需要使用其他地方定义的外部链接变量时，使用extern说明

```
// file1.cpp
int a = 1; // 全局变量定义（默认外部链接）
static int b = 1; // 全局变量定义（static 为内部链接）
extern const int d; // 使用其他文件的常量，用 extern 声明（外部链接）
void print(); // 使用其他文件的函数，直接声明（默认外部链接）
int main() {
    print(); // 1111
}
// file2.cpp
#include <iostream>
const int c = 1; // 全局常量定义（默认内部链接）
extern const int d = 1; // 全局常量定义（extern 为外部链接）
extern int a; // 使用其他文件的变量，用 extern 声明（外部链接）
void print() { // 全局函数定义（默认外部链接）
    int b = 1;
    std::cout << a << b << c << d << std::endl;
}
```

存储期、作用域、链接总结

作用域	说明符	类型	存储期	链接
局部变量	`{ int x{}; }`	定义	自动	无
	`{ static int x{}; }`	定义	**静态**	无
全局变量	`int x{};`	定义	静态	外部
	`static int x{};`	定义	静态	**内部**
	`extern int x;`	**声明**	静态	外部
全局常量	`const int x{};`	定义	静态	内部
	`extern const int x{};`	定义/声明	静态	**外部**
全局函数	`int f();/{}`	定义/声明	-	外部
	`static int f();/{}`	定义/声明	-	**内部**

通过 include 头文件避免不一致

分开编译时，如果定义与声明不一致可能导致问题，如在某源文件中定义`extern const int x=1;`在其他文件声明`extern int x;`则分别可以编译并能链接，但执行`++x;`为未定义行为

函数调用栈

- 程序执行时通常占用四块内存区域
  - 代码区：存放程序的二进制机器代码
  - 静态存储区：存放全局变量、常量、静态变量等
  - 堆区：(后续章节) 存放动态分配的内存
  - 栈区：存放每次函数调用的数据，如形参和局部变量

栈 (stack)

栈是一种**后进先出** (Last In First Out) 的数据结构

- 每次调用函数，会在栈区开辟新空间存放活动记录，包括调用结束后的返回地址与当前函数中的自动变量
  - 栈内存中每次函数调用对应的空间称为一个栈帧 (stack frame)
  - 执行结束后通过返回地址继续执行原函数
- 栈是后进先出，函数调用时 push 进栈，返回时 pop 出栈

函数调用栈实例（程序与输出）

```cpp
#include <iostream>

int f(int x, int y) {
    std::cout << "f.x at " << &x << std::endl;
    std::cout << "f.y at " << &y << std::endl;
    return x * y;
}

int g(int x) {
    int y = 2;
    std::cout << "g.x at " << &x << std::endl;
    std::cout << "g.y at " << &y << std::endl;
    return f(x,y);
}

int main() {
    int x = 32767;
    int y = 32768;
    std::cout << "main.x at " << &x << std::endl;
    std::cout << "main.y at " << &y << std::endl;
    g(x + y);
}
```

程序输出（地址会变）

```
main.x at 0x5ffe5c
main.y at 0x5ffe58
g.x at 0x5ffe30
g.y at 0x5ffe1c
f.x at 0x5ffdf0
f.y at 0x5ffdf8
```

函数调用栈实例（IDE 线程栈）

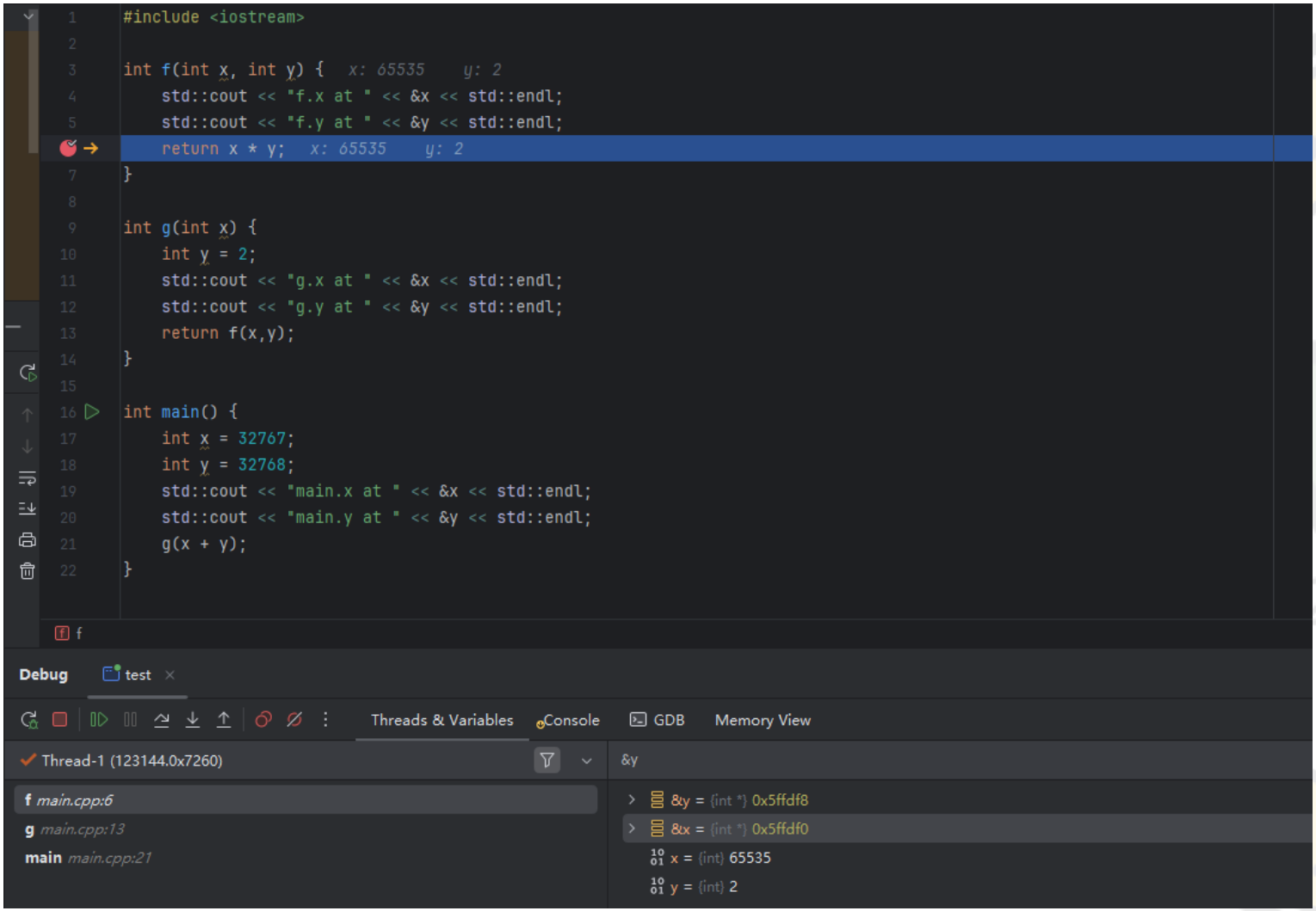


图 2: 函数调用栈，执行至第 6 行时，栈顶为f函数，栈底为main函数

函数调用栈实例（内存视图）

0x00000000005ffdc0	01 00 00 00	00 00 00 00	00 d9 70 e4	f6 7f 00 00	··········p·····
0x00000000005ffdd0	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	················
0x00000000005ffde0	20 fe 5f 00	00 00 00 00	3e 1b 65 e4	f6 7f 00 00	·_·····>·e·····
0x00000000005ffdf0	ff ff 00 00	00 00 00 00	02 00 00 00	00 00 00 00	················
0x00000000005ffe00	01 00 00 00	00 00 00 00	00 d9 70 e4	f6 7f 00 00	··········p·····
0x00000000005ffe10	00 00 00 00	00 00 00 00	00 00 00 00	02 00 00 00	················
0x00000000005ffe20	60 fe 5f 00	00 00 00 00	e2 1b 65 e4	f6 7f 00 00	`·_·····e·····
0x00000000005ffe30	ff ff 00 00	00 00 00 00	a8 b8 65 e4	f6 7f 00 00	··········e·····
0x00000000005ffe40	b0 14 63 00	00 00 00 00	00 00 00 00	01 00 00 00	··c··············
0x00000000005ffe50	00 10 65 e4	f6 7f 00 00	00 80 00 00	ff 7f 00 00	··e··············
0x00000000005ffe60	e0 fe 5f 00	00 00 00 00	40 13 65 e4	f6 7f 00 00	··_·····@·e·····
0x00000000005ffe70	01 00 00 00	f6 7f 00 00	08 c0 73 e4	f6 7f 00 00	··········s·····

图 3: f 栈帧的内存视图 (int 为 4 字节，小端)

- f函数的栈帧在0x5ffde0前后，内容如下
  - 0x5ffde0：保存调用者的栈帧地址，值为0x5ffe20
  - 0x5ffde8：返回地址，值为0x7ff6e4651b3e[8]
  - 0x5ffdf0：f的形参x，值为0x0000ffff，即 65535
  - 0x5ffdf8：f的形参y，值为0x00000002，即 2

[8] g函数开头的地址为0x7ff6e4651aae

函数调用栈实例（内存视图）

0x00000000005ffdc0	01 00 00 00	00 00 00 00	00 d9 70 e4	f6 7f 00 00
0x00000000005ffdd0	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00
0x00000000005ffde0	20 fe 5f 00	00 00 00 00	3e 1b 65 e4	f6 7f 00 00
0x00000000005ffdf0	ff ff 00 00	00 00 00 00	02 00 00 00	00 00 00 00
0x00000000005ffe00	01 00 00 00	00 00 00 00	00 d9 70 e4	f6 7f 00 00
0x00000000005ffe10	00 00 00 00	00 00 00 00	00 00 00 00	02 00 00 00
0x00000000005ffe20	60 fe 5f 00	00 00 00 00	e2 1b 65 e4	f6 7f 00 00
0x00000000005ffe30	ff ff 00 00	00 00 00 00	a8 b8 65 e4	f6 7f 00 00
0x00000000005ffe40	b0 14 63 00	00 00 00 00	00 00 00 00	01 00 00 00
0x00000000005ffe50	00 10 65 e4	f6 7f 00 00	00 80 00 00	ff 7f 00 00
0x00000000005ffe60	e0 fe 5f 00	00 00 00 00	40 13 65 e4	f6 7f 00 00
0x00000000005ffe70	01 00 00 00	f6 7f 00 00	08 c0 73 e4	f6 7f 00 00

图 4: g 栈帧的内存视图 (int 为 4 字节，小端)

- g函数的栈帧在0x5ffe20前后，内容如下
  - 0x5ffe1c：g的局部变量y，值为0x00000002，即 2
  - 0x5ffe20：保存调用者的栈帧地址，值为0x5ffe60
  - 0x5ffe28：返回地址，值为0x7ff6e4651be2[9]
  - 0x5ffe30：g的形参x，值为0x0000ffff，即 65535

[9] main函数开头的地址为0x7ff6e4651b44

函数调用栈实例（内存视图）

0x00000000005ffdc0	01 00 00 00	00 00 00 00	00 d9 70 e4	f6 7f 00 00	··········p·····
0x00000000005ffdd0	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	················
0x00000000005ffde0	20 fe 5f 00	00 00 00 00	3e 1b 65 e4	f6 7f 00 00	·_·····>·e·····
0x00000000005ffdf0	ff ff 00 00	00 00 00 00	02 00 00 00	00 00 00 00	················
0x00000000005ffe00	01 00 00 00	00 00 00 00	00 d9 70 e4	f6 7f 00 00	··········p·····
0x00000000005ffe10	00 00 00 00	00 00 00 00	00 00 00 00	02 00 00 00	················
0x00000000005ffe20	60 fe 5f 00	00 00 00 00	e2 1b 65 e4	f6 7f 00 00	`·_·····e·····
0x00000000005ffe30	ff ff 00 00	00 00 00 00	a8 b8 65 e4	f6 7f 00 00	··········e·····
0x00000000005ffe40	b0 14 65 00	00 00 00 00	00 00 00 00	01 00 00 00	··e··············
0x00000000005ffe50	00 10 65 e4	f6 7f 00 00	00 80 00 00	ff 7f 00 00	··e··············
0x00000000005ffe60	e0 fe 5f 00	00 00 00 00	40 13 65 e4	f6 7f 00 00	··_·····@·e·····
0x00000000005ffe70	01 00 00 00	f6 7f 00 00	08 c0 73 e4	f6 7f 00 00	··········s·····

图 5: main 栈帧的内存视图 (int 为 4 字节，小端)

- main函数的栈帧在0x5ffe60前后，内容如下
  - 0x5ffe58：main的局部变量y，值为0x00008000，即 32768
  - 0x5ffe5c：main的局部变量x，值为0x00007fff，即 32767
  - 0x5ffe60：返回操作系统的地址

inline 说明符

https://en.cppreference.com/cpp/language/inline

- 函数调用需要分配栈内存，进行代码跳转，产生额外的开销

```
double square(double x) { return x * x; }
```

- 将语句直接写到调用处效率更高，但产生代码重复，不符合模块化编程思想
- inline说明符**建议**编译器将函数作为**内联函数**，编译时直接将函数调用“改写”为对应语句

```
inline double square(double x) { return x * x; }
double circle_area(double r) {
    return 3.14 * square(r); // equivalent to writing return 3.14 * r * r;
}
```

- inline会增加程序大小，因此长函数不宜内联
- 该说明符仅为建议，写了也不一定内联，不写也可能内联
- 由于内联需要创建代码副本，必须定义在头文件中，否则其他文件无法编译

其他说明符（拓展）

我们在后续章节中还会接触到其他说明符，总结如下

- 访问控制：`public`✓，`private`✓，`protected` (§9.2)，`friend` (§9.12)
- 存储期：`static`(静态变量✓，静态数据成员与成员函数 §9.14)，`thread_local`(了解)，`auto`×，`register`×
- 链接：`static`✓，`extern`✓
- 常量相关：`const`(成员函数✓，常量变量 §7.4，常量引用 §10.10)，`constexpr`✓，`mutable`(了解)，`volatile`(了解)
- 函数相关：`explicit`✓，`inline`✓，`virtual` (§12.3)

目录

函数与类型转换

随机

说明符 (static, extern, inline)、存储类别、函数调用栈

拷贝与引用

函数重载、默认参数、函数模板

递归与迭代

按值传递

- 目前为止，我们定义函数的形参列表是以下格式

```
int f(int x, int y);
void print(std::string s);
```

- 通过函数调用栈发现，调用函数后内存中有两份变量

```
#include <iostream>
#include <string>
void print(std::string s) { std::cout << s; }
int main() {
    std::string s{"Hello World!"};
    print(s);
}
```

```
0x000000016bcdee50  70 37 32 f1  01 00 00 00  90 f0 cd 6b  01 00 00 00  p72·····k····
0x000000016bcdee60  48 65 6c 6c  6f 20 57 6f  72 6c 64 21  00 00 00 00  Hello World!····
0x000000016bcdee70  58 c0 06 f1  01 00 00 0c  48 65 6c 6c  6f 20 57 6f  X·····Hello Wo
0x000000016bcdee80  72 6c 64 21  00 00 00 00  58 c0 06 f1  01 00 00 0c  rld!····X·····
```

- 这种传递实参的方式称为“按值传递”

按值传递 (Pass by Value)

函数将实参的值**拷贝**到形参并执行函数体，函数内对形参的操作不会影响实参的值，此时形参可以认为是临时变量

按引用传递

- 对简单数据类型 (如int)，按值传递很快；并且安全性较好，不会因为函数内的操作意外改变函数外的值
- 但有些场景，按值传递无法满足要求
  - 有时我们确实想让实参的值受到影响，例如，写程序交换两个数

```
void swap(int x, int y) { // not working!
    int tmp = x; x = y; y = tmp; // only copied values are swapped
}
```

  - 对于大型数据（如对象、长字符串），拷贝的开销很高
- 此时，我们需要“按引用传递”

按引用传递 (Pass by Reference)

形参是函数为实参起的**别名**，函数内可以通过别名直接访问（读写）实参的内存区域

程序五：交换两个变量

```cpp
#include <iostream>

void swap(int& x, int &y) {
    int tmp = x; x = y; y = tmp;
}

int main() {
    int x{1}, y{2};
    swap(x, y);
    std::cout << x << y << std::endl; // 21
}
```

- 按引用传递时，形参格式为类型& 形参名，空格不影响编译
- 建议将`int& x`从右向左读成 “x is a reference to an int” (x 引用一个 int)，对后面的学习有帮助
- （非常数）引用传递的实参必须是左值[10]，不能是临时变量，例如 `swap(x, x+y); // error: cannot bind int& to int`

[10] 能出现在赋值号左边的值

常量引用

- 类似地，std::string& s按引用传递对象，也可以避免拷贝开销
- 然而，这样也允许了函数修改实参的值，不如按值传递安全
- 按常量引用传递可以既避免拷贝（引用），也防止修改（常量）

```
#include <iostream>
#include <string>
void print(const std::string& s) { std::cout << s; }
int main() {
    std::string s{"Hello World!"};
    print(s);
}
```

- 常量引用也可以写成std::string const& x，读作 “x is a reference to a constant string” (x 引用一个不变的 string)
- 注意const是修饰string的，表示字符串 s 在函数内不会被修改
- 习惯将const写在前面，但含义不变

按值、按引用、按常量引用对比

	类型	是否拷贝	是否可修改形参
`int f(int);`	by Value	拷贝	可修改，不影响实参
`int f(int&);`	by Ref	不拷贝	可修改，影响实参
`int f(const int&);` `int f(int const&);`	by Const-Ref	不拷贝	不可修改

- 理论上还存在按常量值传递 (书图 6.19)，但基本没用
- 第 8 章会学习按**地址**传递，与按引用传递基本只是风格差异
- 按引用传递一般要通过地址取变量值，也会有额外开销，因此对于基本类型，尽量用按值传递，只有当需要修改时才用按引用
- 对于`std::string`等大对象，主要使用按引用传递[11]

[11] 后续章节介绍到`std::move`时，我们再讨论大对象的拷贝传递

拷贝与引用（拓展）

类初始化中的拷贝

- 第 3 章讲过用`std::string s{"Hello World"};`初始化对象
- 此处不用 int 初始化的写法，是因为会产生拷贝
- `std::string s = "Hello World";`执行时先构建了一个临时对象，然后用赋值号将它拷贝给了对象 s

变量别名 (Alias)

- 除了形参，引用也可用于给变量起别名，格式为`int &y = x;`
- 此时&号应贴近变量，且必须有初始化

```
int x{0};
int &y{x}, z{x};
++y;
std::cout << x << y << z; // 110
```

函数返回与引用（拓展）

通过引用获得函数的多个返回值

- 当函数有多个返回时，一种做法是将结果存在按值传递的变量里

```
void compute_roots(int x, double& res1, double& res2) {
    res1 = std::sqrt(x); res2 = -res1;
}
int main() {
    double res1, res2;
    compute_roots(3, res1, res2);
}
```

- 但是这种做法不符合 OOP 思想，应为返回结果构建类

返回引用与常量引用

- 函数返回也可以是引用，但不能是自动变量 (结束即销毁)
- 常见于类的成员函数 (如 Getter) 返回大型成员

```
class MyClass {
    std::string message{"Hello!"};
public:
    const std::string& getMessage() const { return message; }
    std::string& getMessage() { return message; } // unsafe! setter is skipped
};
```

目录

函数与类型转换

随机

说明符 (static, extern, inline)、存储类别、函数调用栈

拷贝与引用

函数重载、默认参数、函数模板

递归与迭代

函数重载

- 我们有时需要对不同的类型或数量的参数进行类似的运算，目前的解决方法是为每种计算分别定义一个名字不同的函数

```
int square_int(int x) { return x * x;}
double square_double(double x) { return x * x;}
int max2(int, int);
int max3(int, int, int);
```

- 使用起来不方便，需要记忆多个函数名
- 如果用户不知道有square_int()，直接调用了square_double()，会有性能损失
- C++ 允许函数重名，通过函数签名来区分函数 (见课件第9页)

函数重载 (Function Overloading)

定义函数名相同，但参数类型/数量/顺序 (或 const 说明) 不同的函数

- 返回值类型不是函数签名的一部分，不能通过返回值类型区分
- 尽管没有检查，重载的函数在逻辑上应该进行类似的计算

程序六：重载 square 函数

```cpp
#include <iostream>

int square(int x) {
    std::cout << "square of int " << x << " is ";
    return x * x;
}

double square(double x) {
    std::cout << "square of double " << x << " is ";
    return x * x;
}

int main() {
    std::cout << square(7) << std::endl;
    std::cout << square(7.5) << std::endl;
}
```

- 编译器将形参类型列表编码进汇编函数名，从而选择正确的函数
- 编译器会优先寻找参数类型完全匹配的函数，否则尝试转换
- 如果存在歧义，可能导致编译报错

```cpp
long square(long x);
double square(double x);

int main() {
    square(7); // error: call of overloaded 'square(int)' is ambiguous
}
```

默认参数

默认实参 (Default Arguments)

为部分形参定义默认值，当调用函数实参不足时，用默认值补上

```
1  int volume(int length=1, int width=1, int height=1);
2
3  int main() {
4      volume(); // equivalent to volume(1,1,1)
5      volume(2); // equivalent to volume(2,1,1)
6      volume(2,2); // equivalent to volume(2,2,1)
7      volume(2,2,2);
8  }
9
10 int volume(int length, int width, int height) {
11     return length * width * height;
12 }
```

- 默认参数必须从右向左给，不能是`int f(int x=1, y);`
- 调用时实参从左向右匹配，不够的由默认参数依次补齐
- 默认参数的值需要在函数第一次出现时给出，不能写多次
  - 通常写在头文件的函数声明中，源文件的定义中不再写

默认参数与函数重载的关系

- 默认参数可以认为是一种特殊的函数重载
- 每多一个默认参数都相当于多了一个参数数量不同的重载函数

```
int volume(int length=1, int width=1, int height=1);
```

- 上面的默认参数相当于定义了如下 4 个重载函数，分别接受 3/2/1/0 个变量

```
int volume(int length, int width, int height);
int volume(int length, int width) { return volume(length, width, 1); }
int volume(int length) { return volume(length, 1, 1); }
int volume() { return volume(1, 1, 1); }
```

- 因此，只需对最上面（不带默认参数）的函数提供实现

给了默认参数后，就不能定义具有同样签名的重载函数

函数模板

https://en.cppreference.com/cpp/language/function_template

- 函数重载方便我们使用一样的函数名调用函数，但函数的定义还是需要分别写（尽管他们几乎完全一致）

```
void swap(int& x, int& y) {
    int tmp = x; x = y; y = tmp;
}

void swap(long& x, long& y) {
    long tmp = x; x = y; y = tmp;
}

void swap(double& x, double& y) {
    double tmp = x; x = y; y = tmp;
}
```

- 当重载函数的逻辑完全一致，且形参数量一致，仅有类型不同时，通过函数模板合并其实现

函数模板 (Function Template)

```
template <typename T1, class T2>
函数定义
```

程序六：swap 函数模板

```cpp
#include <iostream>
template <typename T> // equivalent to template <class T>
void swap(T& x, T& y) {
    T tmp{x};
    x = y;
    y = tmp;
}
int main() {
    std::string s{"Hello"}, t{"World\n"};
    swap(t, s); // specialize to swap<std::string>
    std::cout << s << t;
}
```

- 函数模板需要在函数头前加上`template`保留字，以及由尖括号括起的模板参数列表 (template parameter)
- 模板参数列表的参数为`typename` 名字或`class` 名字，二者无区别，多个参数名字不能重复，用逗号分隔
  - 参数为形式类型参数 (formal type parameter)，通常用 1-2 个大写字母
  - 可用于返回类型、形参列表、函数体内变量定义等，代替具体类型
  - 编译器会根据调用者的实参类型，将形式类型参数替换为具体的类型，称为实例化
- （拓展）若返回类型取决于变量，C++11 允尾随返回类型

```cpp
template <typename A, typename B>
auto multiply(const A& a, const B& b) -> decltype(a * b) {
```

函数模板注意点

- 和函数重载的原理一样，每个typename都应在形参列表中出现至少一次，否则无法根据实参确定调用哪个函数

```
template <typename T>
T power(double x, int y) { ... }
int main() { power(1.0, 2); } // error: couldn't deduce template parameter 'T'
```

- 调用时可以用尖括号具体指定实例化的函数

```
int main() { power<double>(1.0, 2); } // OK, instantiated to T=double
```

- 模板函数是**有需要才实例化**，实例化的才编译，依赖使用正确的实参，并且其中的编译错误若无使用可能不会发现

```
#include <string>
template <typename T>
int size(T x) { return x.size(); }
int main() {
    size(std::string{"Hello, World!"}); // OK
    size(1); // error
}
```

- 模板函数定义必须放在头文件中（无法做到接口与实现分离），否则单独编译 cpp 文件时，不知道其他文件需要用到哪些实例
- 类模板、模板特化将在后续章节中介绍

目录

函数与类型转换

随机

说明符 (static, extern, inline)、存储类别、函数调用栈

拷贝与引用

函数重载、默认参数、函数模板

递归与迭代

递归

递归与分治的思想 (Recursion)

将一个大问题转化为数个与原问题相同，但规模更小的问题，直到达到基本情况并解决

- 例如，二分法猜数，是将“猜 1-100 的数”，转化为“猜 1-49”或“猜 51-100”，直到范围内只有一个数，问题解决
- 数学归纳法同样是先证明基本情况，再将对 n 的证明转化为对 $n-1$ 和/或 $n-2$ 的证明
- 在计算机中，递归函数是指直接或间接调用自身的函数
- 以设计阶乘函数`int factorial(unsigned n);`为例
  - 若 $n == 0$ 或 1，则其阶乘为 1
  - 否则，其阶乘为 $n! = (n-1)! * n$

```
unsigned long factorial(unsigned n) {
    return (n <= 1) ? 1L : factorial(n - 1) * n;
}
```

迭代

- 递归的每次调用都会产生新的栈帧，若超过可用内存就会导致栈溢出 (stack overflow)
- 将递归问题转为迭代来实现通常有利于节约内存
  - 递归的基本情况 → 迭代的起点
  - 将大问题转小问题 → 用小问题的结果推导新结果
  - 通过函数调用的隐式循环 → 显式循环

迭代 (Iteration)

从基本情况开始循环，不断推导更大问题的结果，直到目标问题解决

```
unsigned long factorial(unsigned n) {
    unsigned long prod{1};
    for (unsigned i{1}; i <= n; ++i) {
        prod *= i;
    }
    return prod;
}
```

程序七：斐波那契数列

递归法：程序简单容易理解，但时间空间效率低，有重复计算

```
1  unsigned long fibonacci(unsigned n) {
2      if (n == 0) return 0;
3      if (n == 1) return 1;
4      return fibonacci(n - 1) + fibonacci(n - 2);
5  }
```

迭代法[12]：任何能递归解决的问题都能用迭代解决，但代码更难写

```
1  unsigned long fibonacci(unsigned n) {
2      unsigned long a_0 = 0;
3      unsigned long a_1 = 1;
4      bool b{false};
5      for (int i = 2; i <= n; ++i, b=!b) {
6          unsigned long &a_i = b ? a_1 : a_0;
7          a_i += b ? a_0 : a_1;
8      }
9      return b ? a_0 : a_1;
10 }
```

数学法：对问题进行充分的数学优化往往有助于程序设计

```
1  unsigned long fibonacci(unsigned n) {
2      return (std::pow((1 + std::sqrt(5))/2, n) - std::pow((1 - std::sqrt(5))/2, n)) / std::sqrt(5);
3  }
```

[12]本问题的迭代法称为**动态规划** (dynamic programming)，是一种记录中间结果，避免重复计算的算法

总结

- 理解程序模块的设计和构建✓
- 熟练掌握 C++ 中函数相关功能✓
  - 标准库函数✓
  - 隐式类型转换✓
  - 程序员定义的函数✓
    - 随机
    - 存储期、作用域、链接
    - 函数调用栈
    - 内联
    - 函数重载（含默认参数与函数模板）
- 递归✓
- 按值调用和按引用调用✓